

IDENTIFICACIÓN Y SELECCIÓN DE HERRAMIENTAS DE VERSIONAMIENTO

Proyecto Formativo

Construcción de software integrador de tecnologías orientadas a servicios

Programa de Formación: Análisis y Desarrollo de Software

Evidencia: GA7-220501096-AA1-EV01 – Informe técnico de plan de trabajo para construcción de software.

Proyecto de Software: Plataforma Web para la gestión comercial de la marca Ukiyo Co.

IDENTIFICACIÓN Y SELECCIÓN DE HERRAMIENTAS DE VERSIONAMIENTO

Proyecto: Desarrollo de plataforma web para Ukiyo CO

1. Introducción

El desarrollo de software requiere mecanismos que permitan controlar las modificaciones realizadas sobre el código fuente durante las diferentes etapas de construcción. A medida que un proyecto aumenta su número de archivos, funcionalidades y versiones, resulta necesario disponer de herramientas que permitan registrar los cambios, recuperar estados anteriores y mantener una estructura organizada del código.

Las herramientas de control de versiones permiten gestionar diferentes estados de un proyecto y mantener un historial de las modificaciones realizadas. Estas herramientas son especialmente importantes en proyectos de desarrollo web, debido a que durante la construcción de una aplicación se realizan cambios constantes sobre archivos, componentes, funcionalidades y configuraciones.

En el presente documento se identifican y clasifican diferentes herramientas de versionamiento local y remoto, con el propósito de seleccionar aquellas que resulten apropiadas para el desarrollo de la plataforma web de Ukiyo CO.

Para el proyecto se selecciona Git como herramienta principal de control de versiones local y GitHub como plataforma de almacenamiento y gestión del repositorio remoto. Esta combinación permitirá mantener un historial de cambios, trabajar con diferentes ramas y conservar una copia remota del proyecto.

2. Objetivos

2.1 Objetivo general

Identificar, comparar y seleccionar herramientas de versionamiento local y remoto apropiadas para el control de cambios del código fuente durante el desarrollo de la plataforma web de Ukiyo CO.

2.2 Objetivos específicos

1. Identificar diferentes herramientas utilizadas para el control de versiones en proyectos de software.
2. Clasificar las herramientas de versionamiento de acuerdo con su funcionamiento local o remoto.
3. Describir las principales características de Git como herramienta de versionamiento local.
4. Describir las principales características de GitHub como plataforma de versionamiento y almacenamiento remoto.
5. Comparar las características de las herramientas seleccionadas.
6. Establecer una estrategia básica de versionamiento para el proyecto Ukiyo CO.

3. Contexto del proyecto Ukiyo CO

Ukiyo CO es una propuesta de marca de ropa orientada al segmento de moda urbana y streetwear. Dentro del proyecto formativo de Análisis y Desarrollo de Software se plantea la construcción de una plataforma web que permita fortalecer la presencia digital de la marca y apoyar procesos relacionados con la comercialización de sus productos.

La plataforma contempla funcionalidades como:

- Página principal.
- Catálogo de productos.
- Consulta de disponibilidad.
- Detalle de productos.
- Carrito de compras.
- Registro de usuarios.
- Inicio de sesión.
- Generación de pedidos.
- Gestión de inventario.
- Administración de productos.
- Gestión de usuarios.
- Gestión de pedidos.
- Comunicación con la marca.

Debido a que el desarrollo de estas funcionalidades implica realizar modificaciones constantes sobre el código fuente, se requiere implementar una estrategia de control de versiones.

4. Concepto de control de versiones

El control de versiones es un sistema que permite registrar y administrar los diferentes cambios realizados sobre los archivos de un proyecto.

Su utilización permite conocer qué modificaciones fueron realizadas, cuándo se realizaron y, dependiendo de la herramienta utilizada, quién las realizó.

Entre las principales funciones de un sistema de control de versiones se encuentran:

- Registrar modificaciones.
- Mantener un historial.
- Recuperar versiones anteriores.
- Crear ramas de desarrollo.
- Comparar cambios.
- Integrar modificaciones.
- Facilitar la colaboración.
- Mantener copias del proyecto.

Git es un sistema distribuido de control de versiones que permite mantener un historial completo del proyecto de manera local. Su arquitectura distribuida permite que cada repositorio tenga una copia del historial del proyecto (Git, s. f.).

5. Clasificación de herramientas de versionamiento

Las herramientas de versionamiento pueden clasificarse, de manera general, en herramientas orientadas al control local del código y plataformas que permiten almacenar y administrar repositorios remotamente.

5.1 Herramientas de versionamiento local

Son herramientas utilizadas directamente desde el equipo del desarrollador para controlar los cambios realizados sobre el código.

Algunos ejemplos son:

- Git.
- Apache Subversion (SVN).
- Mercurial.

5.2 Herramientas de versionamiento remoto

Son plataformas que permiten almacenar repositorios en servidores remotos y proporcionar funcionalidades adicionales para la colaboración y administración del proyecto.

Algunos ejemplos son:

- GitHub.
- GitLab.
- Bitbucket.

Es importante aclarar que Git y GitHub no son herramientas equivalentes. Git es el sistema de control de versiones, mientras que GitHub es una plataforma que permite alojar repositorios Git y proporcionar herramientas adicionales para su administración y colaboración.

6. Herramientas de versionamiento local

6.1 Git

Git es un sistema distribuido de control de versiones utilizado para realizar seguimiento de los cambios efectuados sobre un proyecto de software.

Entre sus principales características se encuentran:

- Control distribuido.
- Funcionamiento local.
- Registro de commits.
- Creación y administración de ramas.
- Comparación de cambios.
- Recuperación de versiones anteriores.
- Integración con plataformas remotas.
- Funcionamiento mediante línea de comandos o interfaces gráficas.

Git permite crear un repositorio local utilizando:

```
git init
```

Después de modificar los archivos del proyecto, los cambios pueden ser preparados mediante:

```
git add .
```

Posteriormente se registra una nueva versión:

```
git commit -m "feat: crear estructura inicial"
```

Finalmente, los cambios pueden enviarse a un repositorio remoto:

```
git push
```

Estos comandos forman parte del flujo básico de trabajo utilizado para registrar y sincronizar modificaciones.

6.2 Apache Subversion

Apache Subversion, conocido como SVN, es un sistema de control de versiones utilizado para administrar cambios realizados sobre archivos y directorios.

A diferencia de Git, SVN utiliza una arquitectura centralizada en la que existe un repositorio principal desde el cual los usuarios obtienen y envían modificaciones.

Entre sus características se encuentran:

- Arquitectura centralizada.
- Control de versiones.
- Gestión de archivos y directorios.
- Registro histórico.
- Control de acceso.
- Soporte para proyectos colaborativos.

Aunque SVN es una herramienta válida para el control de versiones, para el proyecto Ukiyo CO se seleccionará Git debido a su modelo distribuido y su integración con plataformas como GitHub.

6.3 Mercurial

Mercurial es otro sistema distribuido de control de versiones. Al igual que Git, permite mantener repositorios locales y registrar diferentes estados del código.

Entre sus características se encuentran:

- Arquitectura distribuida.
- Control de versiones.
- Repositorios locales.
- Creación de ramas.

- Registro de cambios.
- Capacidad para trabajar con repositorios remotos.

Para el proyecto Ukiyo CO no será seleccionado debido a que Git ofrece una integración más conveniente con el flujo de trabajo planteado y con la plataforma GitHub.

7. Herramientas de versionamiento remoto

7.1 GitHub

GitHub es una plataforma utilizada para alojar y gestionar repositorios Git. Permite almacenar el código fuente de manera remota y proporciona funcionalidades relacionadas con colaboración, ramas, seguimiento de cambios y revisión de código.

Entre sus principales características se encuentran:

- Repositorios remotos.
- Integración con Git.
- Ramas.
- Pull requests.
- Historial de cambios.
- Gestión colaborativa.
- Seguimiento de incidencias.
- Documentación mediante archivos README.
- Integración con herramientas externas.

Para el proyecto Ukiyo CO, GitHub será utilizado como repositorio remoto principal.

7.2 GitLab

GitLab es una plataforma que permite administrar repositorios Git y proporciona herramientas relacionadas con el desarrollo y ciclo de vida del software.

Entre sus funcionalidades se encuentran:

- Repositorios Git.
- Ramas.
- Merge requests.
- Seguimiento de incidencias.
- Automatización mediante CI/CD.

- Gestión de proyectos.

GitLab representa una alternativa viable para el almacenamiento remoto del proyecto, aunque para Ukiyo CO se seleccionará GitHub debido a su facilidad de integración con el entorno de trabajo propuesto.

7.3 Bitbucket

Bitbucket es una plataforma de alojamiento de repositorios que permite trabajar con Git y gestionar proyectos de desarrollo.

Entre sus características se encuentran:

- Repositorios remotos.
- Control de acceso.
- Ramas.
- Pull requests.
- Integración con herramientas de desarrollo.
- Funcionalidades orientadas a equipos.

Al igual que GitLab, puede utilizarse como alternativa para el almacenamiento remoto, pero no será la plataforma seleccionada para el proyecto Ukiyo CO.

8. Comparación de herramientas

Herramienta	Tipo	Características principales	Seleccionada
Git	Local	Control distribuido, commits, ramas, historial	Sí
SVN	Local/centralizado	Repositorio central, control de versiones	No
Mercurial	Local/distribuido	Control distribuido y repositorios locales	No
GitHub	Remoto	Repositorios, ramas, colaboración	Sí
GitLab	Remoto	Repositorios, CI/CD, colaboración	No
Bitbucket	Remoto	Repositorios, ramas, colaboración	No

La selección de Git y GitHub se realiza considerando las necesidades del proyecto, la facilidad de integración entre ambas herramientas y la disponibilidad de recursos para el desarrollo.

9. Diferencias entre versionamiento local y remoto

Característica	Versionamiento local	Versionamiento remoto
Ubicación	Equipo del desarrollador	Servidor externo
Ejemplo	Git	GitHub
Historial	Sí	Sí
Ramas	Sí	Sí
Copia remota	No necesariamente	Sí
Trabajo sin Internet	Sí, en gran medida	Limitado para sincronización
Colaboración	Más limitada	Facilita colaboración
Respaldo externo	No necesariamente	Sí

El versionamiento local permite trabajar directamente sobre el proyecto y registrar modificaciones sin depender de una conexión permanente. El versionamiento remoto permite mantener una copia externa del proyecto y facilita la colaboración y sincronización entre diferentes entornos.

Ambos sistemas pueden utilizarse de manera complementaria.

10. Herramientas seleccionadas para Ukiyo CO

Después de analizar las diferentes alternativas, se seleccionan:

Herramienta local

Git

Se utilizará para:

- Crear el repositorio local.
- Registrar commits.
- Crear ramas.
- Comparar cambios.
- Recuperar versiones anteriores.
- Preparar cambios para ser enviados al repositorio remoto.

Herramienta remota

GitHub

Se utilizará para:

- Almacenar el repositorio remoto.
- Mantener una copia del proyecto.
- Administrar ramas.
- Revisar cambios.
- Documentar el proyecto.
- Facilitar la colaboración.

La combinación de ambas herramientas permitirá establecer un flujo de trabajo organizado.

11. Configuración inicial de Git

Para iniciar el control de versiones del proyecto se deberá crear el repositorio local.

Desde la carpeta principal del proyecto se ejecutará:

```
git init
```

Posteriormente se deberá verificar el estado del repositorio:

```
git status
```

Los archivos podrán ser agregados al área de preparación:

```
git add .
```

El primer commit podrá registrarse mediante:

```
git commit -m "chore: crear estructura inicial del proyecto"
```

Después se deberá crear el repositorio correspondiente en GitHub y establecer su conexión con el repositorio local.

El repositorio remoto podrá configurarse mediante:

```
git remote add origin URL_DEL_REPOSITORIO
```

Posteriormente se enviará la rama principal:

```
git push -u origin main
```

De esta manera, el proyecto local quedará conectado con su repositorio remoto.

12. Estrategia de ramas

Para mantener organizada la construcción del software se utilizará una estructura básica de ramas.

```
main
|
├── feature/catalogo
├── feature/productos
├── feature/carrito
├── feature/login
├── feature/inventario
└── feature/admin
```

La rama main representará la versión estable del proyecto.

Las ramas feature estarán destinadas al desarrollo de funcionalidades específicas.

Por ejemplo:

```
git checkout -b feature/catalogo
```

Después de realizar los cambios:

```
git add .
git commit -m "feat: desarrollar catálogo de productos"
```

Finalmente, la rama podrá ser enviada al repositorio remoto:

```
git push -u origin feature/catalogo
```

Esta estrategia permitirá mantener separados los cambios realizados sobre diferentes funcionalidades.

13. Flujo de trabajo propuesto

El flujo de trabajo para Ukiyo CO será:

Análisis de requerimiento



Creación de rama



Este procedimiento permitirá mantener un historial organizado y reducir el riesgo de afectar directamente la versión estable del proyecto.

14. Aplicación al proyecto Ukiyo CO

La estrategia de versionamiento se aplicará a los diferentes módulos de la plataforma.

Por ejemplo, durante el desarrollo del catálogo se creará una rama denominada:

feature/catalogo

Los cambios relacionados con el carrito se desarrollarán en:

feature/carrito

Las funcionalidades administrativas podrán desarrollarse en:

feature/admin

Cada funcionalidad será desarrollada y probada de manera independiente antes de integrarse a la versión principal.

Los commits deberán describir claramente las modificaciones realizadas.

Ejemplos:

feat: crear estructura del catálogo
feat: agregar tarjetas de productos
feat: implementar carrito
fix: corregir cálculo del total
style: ajustar diseño responsive
docs: actualizar documentación

15. Ventajas de la estrategia seleccionada

La utilización de Git y GitHub proporciona diferentes ventajas para el proyecto.

Trazabilidad

Permite consultar el historial de cambios realizados sobre el código.

Recuperación

Permite regresar a versiones anteriores cuando una modificación genere errores.

Organización

Las ramas permiten trabajar diferentes funcionalidades de manera independiente.

Respaldo

GitHub proporciona un repositorio remoto donde se almacena una copia del proyecto.

Colaboración

En futuras etapas, otros desarrolladores podrán participar en el proyecto utilizando el mismo repositorio.

Mantenimiento

El historial de commits facilita identificar cuándo y por qué se realizaron determinadas modificaciones.

16. Recomendaciones para el control de versiones

Para mantener un repositorio organizado se establecen las siguientes recomendaciones:

1. Realizar commits frecuentes y relacionados con cambios concretos.

2. Utilizar mensajes descriptivos.
3. Evitar almacenar archivos innecesarios.
4. No incluir contraseñas ni claves privadas en el repositorio.
5. Utilizar ramas para desarrollar nuevas funcionalidades.
6. Realizar pruebas antes de integrar cambios a main.
7. Mantener actualizado el repositorio remoto.
8. Documentar el proyecto mediante un archivo README.md.
9. Evitar realizar modificaciones directamente sobre la versión estable cuando sea posible.
10. Revisar los cambios antes de realizar un commit.

17. Resultado esperado

Con la implementación de Git y GitHub se espera establecer un sistema organizado para administrar las diferentes versiones de la plataforma web de Ukiyo CO.

El proyecto contará con un repositorio local encargado de controlar las modificaciones realizadas durante el desarrollo y un repositorio remoto destinado a almacenar y respaldar el código.

La estrategia permitirá mantener un historial de cambios y facilitar la integración progresiva de funcionalidades como catálogo, productos, carrito, usuarios, inventario y administración.

18. Conclusiones

El análisis realizado permite establecer que las herramientas de control de versiones son fundamentales para mantener organizado el desarrollo de una aplicación de software. Su utilización permite registrar modificaciones, mantener diferentes versiones y recuperar estados anteriores del proyecto.

Para el desarrollo de la plataforma web de Ukiyo CO se seleccionó Git como herramienta de versionamiento local debido a sus características de sistema distribuido, capacidad para administrar ramas y registro de cambios mediante commits.

Como herramienta de versionamiento remoto se seleccionó GitHub, debido a que permite alojar repositorios Git y proporciona funcionalidades para administrar ramas, revisar cambios, documentar proyectos y facilitar la colaboración.

La combinación de Git y GitHub permitirá establecer un flujo de trabajo organizado para el desarrollo de Ukiyo CO, diferenciando claramente las actividades realizadas en el entorno local y las operaciones realizadas sobre el repositorio remoto.

Finalmente, la implementación de una estrategia basada en ramas permitirá desarrollar funcionalidades de manera independiente y posteriormente integrarlas a una versión estable del proyecto, contribuyendo al control y mantenimiento del código fuente.

19. Referencias

Git. (s. f.). *Git documentation*. <https://git-scm.com/docs>

GitHub. (s. f.). *GitHub Docs*. <https://docs.github.com/>

GitHub. (s. f.). *About repositories*. GitHub Docs.
<https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>

GitLab. (s. f.). *GitLab documentation*. <https://docs.gitlab.com/>

Apache Software Foundation. (s. f.). *Apache Subversion documentation*.
<https://subversion.apache.org/docs/>

Mercurial. (s. f.). *Mercurial SCM*. <https://www.mercurial-scm.org/>